



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και
Μηχανικών Υπολογιστών (ΣΗΜΜΥ)

8ο εξάμηνο

Ψηφιακά Συστήματα VLSI

4η εργαστηριακή άσκηση

Δημήτρης Βατάλας

(Α.Μ.: 03121093)

Αριστοτέλης Γεώργιος Δρίτσας

(Α.Μ.: 03122127)

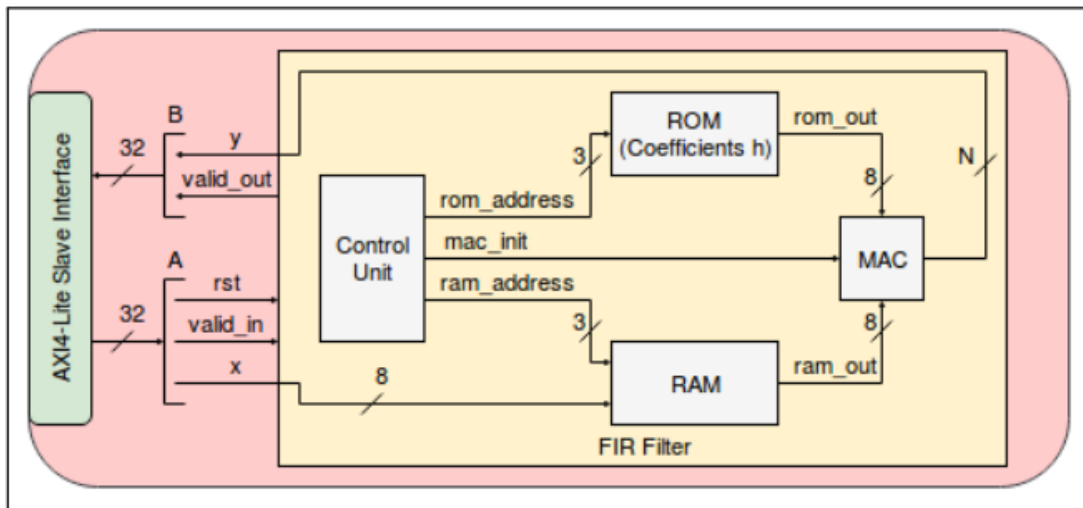
Αθήνα, Μάρτιος 2026

Contents

Εισαγωγή	2
AXI4-Lite IP	2
Κώδικας AXI4-Lite interface	2
Κώδικας control-unit	10
Κώδικας MAC	12
Υλοποίηση συστήματος και παραγωγή bitstream	13
Προγραμματισμός του ZYNQ SoC FPGA και εκτέλεση της εφαρμογής	14

Εισαγωγή

Στην παρούσα εργαστηριακή άσκηση έγινε προσαρμογή του κώδικα της προηγούμενης άσκησης, ο οποίος υλοποιεί ένα φίλτρο FIR, με στόχο την υλοποίησή του στην αναπτυξιακή πλακέτα ZYBO και τη λειτουργία του σε πραγματικό hardware. Παράλληλα, υλοποιήθηκε κατάλληλος κώδικας για τον ενσωματωμένο επεξεργαστή ARM, ώστε τα δεδομένα εισόδου να αποστέλλονται από αυτόν προς το FPGA για επεξεργασία, ενώ τα αποτελέσματα επιστρέφουν πίσω ακολουθώντας την αντίστροφη διαδρομή. Τέλος η επικοινωνία μεταξύ ARM και FPGA θα υλοποιηθεί μέσω διεπαφής AXI4-Lite και το φίλτρο έχει την ακόλουθη μορφή:



AXI4-Lite IP

Αρχικά, δημιουργήσαμε το block design, στο οποίο ενσωματώσαμε το ZYNQ Processing System, ένα custom IP με διεπαφή AXI4-Lite στο οποίο υλοποιείται το φίλτρο FIR, καθώς και το ILA (ώστε σε περίπτωση προβλημάτων να είναι δυνατή η παρατήρηση του AXI Bus.) Οι κώδικες για το custom IP με τη διεπαφή AXI4-Lite παρουσιάζονται παρακάτω:

Κώδικας AXI4-Lite interface

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity myip_final_v1_0_S00_AXI is
  generic (
    -- Users to add parameters here

    -- User parameters ends
    -- Do not modify the parameters beyond this line

    -- Width of S_AXI data bus
    C_S_AXI_DATA_WIDTH    : integer    := 32;
    -- Width of S_AXI address bus
    C_S_AXI_ADDR_WIDTH    : integer    := 4
  );
  port (
    -- Users to add ports here
```

```

-- User ports ends
-- Do not modify the ports beyond this line

-- Global Clock Signal
S_AXI_ACLK      : in std_logic;
-- Global Reset Signal. This Signal is Active LOW
S_AXI_ARESETN   : in std_logic;
-- Write address (issued by master, accepted by Slave)
S_AXI_AWADDR    : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
-- Write channel Protection type. This signal indicates the
--   privilege and security level of the transaction, and whether
--   the transaction is a data access or an instruction access.
S_AXI_AWPROT    : in std_logic_vector(2 downto 0);
-- Write address valid. This signal indicates that the master signaling
--   valid write address and control information.
S_AXI_AWVALID   : in std_logic;
-- Write address ready. This signal indicates that the slave is ready
--   to accept an address and associated control signals.
S_AXI_AWREADY   : out std_logic;
-- Write data (issued by master, accepted by Slave)
S_AXI_WDATA     : in std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
-- Write strobes. This signal indicates which byte lanes hold
--   valid data. There is one write strobe bit for each eight
--   bits of the write data bus.
S_AXI_WSTRB     : in std_logic_vector((C_S_AXI_DATA_WIDTH/8)-1 downto 0);
-- Write valid. This signal indicates that valid write
--   data and strobes are available.
S_AXI_WVALID    : in std_logic;
-- Write ready. This signal indicates that the slave
--   can accept the write data.
S_AXI_WREADY    : out std_logic;
-- Write response. This signal indicates the status
--   of the write transaction.
S_AXI_BRESP     : out std_logic_vector(1 downto 0);
-- Write response valid. This signal indicates that the channel
--   is signaling a valid write response.
S_AXI_BVALID    : out std_logic;
-- Response ready. This signal indicates that the master
--   can accept a write response.
S_AXI_BREADY    : in std_logic;
-- Read address (issued by master, accepted by Slave)
S_AXI_ARADDR    : in std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
-- Protection type. This signal indicates the privilege
--   and security level of the transaction, and whether the
--   transaction is a data access or an instruction access.
S_AXI_ARPROT    : in std_logic_vector(2 downto 0);
-- Read address valid. This signal indicates that the channel
--   is signaling valid read address and control information.
S_AXI_ARVALID   : in std_logic;
-- Read address ready. This signal indicates that the slave is
--   ready to accept an address and associated control signals.
S_AXI_ARREADY   : out std_logic;
-- Read data (issued by slave)
S_AXI_RDATA     : out std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
-- Read response. This signal indicates the status of the
--   read transfer.
S_AXI_RRESP     : out std_logic_vector(1 downto 0);
-- Read valid. This signal indicates that the channel is
--   signaling the required read data.

```

```

        S_AXI_RVALID      : out std_logic;
        -- Read ready. This signal indicates that the master can
        -- accept the read data and response information.
        S_AXI_RREADY      : in std_logic
    );
end myip_final_v1_0_S00_AXI;

architecture arch_imp of myip_final_v1_0_S00_AXI is

    -- AXI4LITE signals
    signal axi_awaddr      : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
    signal axi_awready     : std_logic;
    signal axi_wready     : std_logic;
    signal axi_bresp       : std_logic_vector(1 downto 0);
    signal axi_bvalid     : std_logic;
    signal axi_araddr      : std_logic_vector(C_S_AXI_ADDR_WIDTH-1 downto 0);
    signal axi_arready     : std_logic;
    signal axi_rdata       : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal axi_rresp       : std_logic_vector(1 downto 0);
    signal axi_rvalid     : std_logic;

    -- Example-specific design signals
    -- local parameter for addressing 32 bit / 64 bit C_S_AXI_DATA_WIDTH
    -- ADDR_LSB is used for addressing 32/64 bit registers/memories
    -- ADDR_LSB = 2 for 32 bits (n downto 2)
    -- ADDR_LSB = 3 for 64 bits (n downto 3)
    constant ADDR_LSB      : integer := (C_S_AXI_DATA_WIDTH/32)+ 1;
    constant OPT_MEM_ADDR_BITS : integer := 1;

    -----
    ---- Signals for user logic register space example
    -----

    ---- Number of Slave Registers 4
    signal slv_reg0        : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg1        : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg2        : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg3        : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal slv_reg_rden     : std_logic;
    signal slv_reg_wren     : std_logic;
    signal reg_data_out     : std_logic_vector(C_S_AXI_DATA_WIDTH-1 downto 0);
    signal byte_index      : integer;
    signal aw_en           : std_logic;

    signal valid : std_logic;
    signal outp  : std_logic_vector(18 downto 0);

    component fir is
        Port (
            clk: in std_logic;
            rst: in std_logic;
            valid_in: in std_logic;
            x: in std_logic_vector (7 downto 0);
            valid_out: out std_logic;
            y: out std_logic_vector (18 downto 0)
        );
    end component;

begin

    FIR_axi: fir

```

```

port map (
    clk      => S_AXI_ACLK,
    rst      => slv_reg0(9),
    valid_in => slv_reg0(8),
    x        => slv_reg0(7 downto 0),
    valid_out => valid,
    y        => outp
);

-- outp <= slv_reg1(18 downto 0);
-- valid <= slv_reg1(19);
-- slv_reg1(31 downto 20) <= (others => '0');

-- I/O Connections assignments
S_AXI_AWREADY <= axi_awready;
S_AXI_WREADY <= axi_wready;
S_AXI_BRESP <= axi_bresp;
S_AXI_BVALID <= axi_bvalid;
S_AXI_ARREADY <= axi_arready;
S_AXI_RDATA <= axi_rdata;
S_AXI_RRESP <= axi_rresp;
S_AXI_RVALID <= axi_rvalid;
-- Implement axi_awready generation
-- axi_awready is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_awready is
-- de-asserted when reset is low.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_awready <= '0';
            aw_en <= '1';
        else
            if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1' and aw_en = '1')
            then
                -- slave is ready to accept write address when
                -- there is a valid write address and write data
                -- on the write address and data bus. This design
                -- expects no outstanding transactions.
                axi_awready <= '1';
                aw_en <= '0';
            elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then
                aw_en <= '1';
                axi_awready <= '0';
            else
                axi_awready <= '0';
            end if;
        end if;
    end if;
end process;

-- Implement axi_awaddr latching
-- This process is used to latch the address when both
-- S_AXI_AWVALID and S_AXI_WVALID are valid.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then

```

```

    if S_AXI_ARESETN = '0' then
        axi_awaddr <= (others => '0');
    else
        if (axi_awready = '0' and S_AXI_AWVALID = '1' and S_AXI_WVALID = '1' and aw_en = '1')
            then
                -- Write Address latching
                axi_awaddr <= S_AXI_AWADDR;
            end if;
        end if;
    end if;
end process;

-- Implement axi_wready generation
-- axi_wready is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_AWVALID and S_AXI_WVALID are asserted. axi_wready is
-- de-asserted when reset is low.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_wready <= '0';
        else
            if (axi_wready = '0' and S_AXI_WVALID = '1' and S_AXI_AWVALID = '1' and aw_en = '1')
                then
                    -- slave is ready to accept write data when
                    -- there is a valid write address and write data
                    -- on the write address and data bus. This design
                    -- expects no outstanding transactions.
                    axi_wready <= '1';
                else
                    axi_wready <= '0';
                end if;
            end if;
        end if;
    end process;

-- Implement memory mapped register select and write logic generation
-- The write data is accepted and written to memory mapped registers when
-- axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted. Write strobes are
-- used to
-- select byte enables of slave registers while writing.
-- These registers are cleared when reset (active low) is applied.
-- Slave register write enable is asserted when valid address and data are available
-- and the slave is ready to accept the write address and write data.
slv_reg_wren <= axi_wready and S_AXI_WVALID and axi_wready and S_AXI_AWVALID;

process (S_AXI_ACLK)
variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            slv_reg0 <= (others => '0');
            slv_reg2 <= (others => '0');
            slv_reg3 <= (others => '0');
        else
            loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
            if (slv_reg_wren = '1') then
                case loc_addr is
                    when b"00" =>

```

```

    for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
        if ( S_AXI_WSTRB(byte_index) = '1' ) then
            -- Respective byte enables are asserted as per write strobes
            -- slave registor 0
            slv_reg0(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7
            ↪ downto byte_index*8);
        end if;
    end loop;
when b"01" =>
    null;
when b"10" =>
    for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
        if ( S_AXI_WSTRB(byte_index) = '1' ) then
            -- Respective byte enables are asserted as per write strobes
            -- slave registor 2
            slv_reg2(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7
            ↪ downto byte_index*8);
        end if;
    end loop;
when b"11" =>
    for byte_index in 0 to (C_S_AXI_DATA_WIDTH/8-1) loop
        if ( S_AXI_WSTRB(byte_index) = '1' ) then
            -- Respective byte enables are asserted as per write strobes
            -- slave registor 3
            slv_reg3(byte_index*8+7 downto byte_index*8) <= S_AXI_WDATA(byte_index*8+7
            ↪ downto byte_index*8);
        end if;
    end loop;
when others =>
    slv_reg0 <= slv_reg0;
    slv_reg2 <= slv_reg2;
    slv_reg3 <= slv_reg3;
end case;
else
    slv_reg0(8) <= '0';
end if;
end if;
end if;
end process;

-- Implement write response logic generation
-- The write response and response valid signals are asserted by the slave
-- when axi_wready, S_AXI_WVALID, axi_wready and S_AXI_WVALID are asserted.
-- This marks the acceptance of address and indicates the status of
-- write transaction.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_bvalid <= '0';
            axi_bresp <= "00"; --need to work more on the responses
        else
            if (axi_awready = '1' and S_AXI_AWVALID = '1' and axi_wready = '1' and S_AXI_WVALID =
            ↪ '1' and axi_bvalid = '0') then
                axi_bvalid <= '1';
                axi_bresp <= "00";
            elsif (S_AXI_BREADY = '1' and axi_bvalid = '1') then --check if bready is asserted
            ↪ while bvalid is high)

```

```

        axi_bvalid <= '0';                                -- (there is a possibility that
        ↪ bready is always asserted high)
    end if;
end if;
end if;
end process;

-- Implement axi_arready generation
-- axi_arready is asserted for one S_AXI_ACLK clock cycle when
-- S_AXI_ARVALID is asserted. axi_arready is
-- de-asserted when reset (active low) is asserted.
-- The read address is also latched when S_AXI_ARVALID is
-- asserted. axi_araddr is reset to zero on reset assertion.

process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_arready <= '0';
            axi_araddr <= (others => '1');
        else
            if (axi_arready = '0' and S_AXI_ARVALID = '1') then
                -- indicates that the slave has accepted the valid read address
                axi_arready <= '1';
                -- Read Address latching
                axi_araddr <= S_AXI_ARADDR;
            else
                axi_arready <= '0';
            end if;
        end if;
    end if;
end process;

-- Implement axi_rvalid generation
-- axi_rvalid is asserted for one S_AXI_ACLK clock cycle when both
-- S_AXI_ARVALID and axi_arready are asserted. The slave registers
-- data are available on the axi_rdata bus at this instance. The
-- assertion of axi_rvalid marks the validity of read data on the
-- bus and axi_rresp indicates the status of read transaction. axi_rvalid
-- is deasserted on reset (active low). axi_rresp and axi_rdata are
-- cleared to zero on reset (active low).
process (S_AXI_ACLK)
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            axi_rvalid <= '0';
            axi_rresp <= "00";
        else
            if (axi_arready = '1' and S_AXI_ARVALID = '1' and axi_rvalid = '0') then
                -- Valid read data is available at the read data bus
                axi_rvalid <= '1';
                axi_rresp <= "00"; -- 'OKAY' response
            elsif (axi_rvalid = '1' and S_AXI_RREADY = '1') then
                -- Read data is accepted by the master
                axi_rvalid <= '0';
            end if;
        end if;
    end if;
end process;

```

```

-- Implement memory mapped register select and read logic generation
-- Slave register read enable is asserted when valid address is available
-- and the slave is ready to accept the read address.
slv_reg_rden <= axi_arready and S_AXI_ARVALID and (not axi_rvalid);

process (slv_reg0, slv_reg1, slv_reg2, slv_reg3, axi_araddr, S_AXI_ARESETN, slv_reg_rden)
variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    -- Address decoding for reading registers
    loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
    case loc_addr is
        when b"00" =>
            null;
        when b"01" =>
            reg_data_out <= slv_reg1;
        when b"10" =>
            reg_data_out <= slv_reg2;
        when b"11" =>
            reg_data_out <= slv_reg3;
        when others =>
            reg_data_out <= (others => '0');
    end case;
end process;

-- Output register or memory read data
process (S_AXI_ACLK) is
begin
    if (rising_edge(S_AXI_ACLK)) then
        if (S_AXI_ARESETN = '0') then
            axi_rdata <= (others => '0');
        else
            if (slv_reg_rden = '1') then
                -- When there is a valid read address (S_AXI_ARVALID) with
                -- acceptance of read address by the slave (axi_arready),
                -- output the read data
                -- Read address mux
                axi_rdata <= reg_data_out;      -- register read data
            end if;
        end if;
    end if;
end process;

process (S_AXI_ACLK) is
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            slv_reg1 <= (others => '0');
        elsif valid = '1' then
            slv_reg1 <= (31 downto 20 => '0') & valid & outp;
        end if;
    end if;
end process;

end arch_imp;

```

Στον κώδικα της διεπαφής AXI4-Lite αρχικά ενσωματώθηκε το component του FIR φίλτρου στο σύστημα και υλοποιήθηκε το intended port map σύμφωνα με την εκφώνηση όπως φαίνεται παρακάτω:

- $A[7:0] = x$
- $A[8] = \text{valid_in}$
- $A[9] = \text{rst}$
- $A[31:10] = \text{not used}$
- $B[N-1:0] = y$
- $B[N] = \text{valid_out}$
- $B[31:N+1] = \text{not used}$

(Στην υλοποίησή μας, το A αντιστοιχεί στο slv_reg0 και το B στο slv_reg1.)

Στη συνέχεια, πραγματοποιήθηκαν επιπλέον τροποποιήσεις με στόχο τη διασφάλιση της ορθής λειτουργίας των σημάτων valid_in και valid_out. Πιο συγκεκριμένα, στον κώδικα της διεπαφής AXI4-Lite:

- αποτρέψαμε την εγγραφή στον slv_reg1, ο οποίος αντιστοιχεί στην έξοδο του φίλτρου,
- χειριστήκαμε κατάλληλα το σήμα valid_in, ώστε να μηδενίζεται αυτόματα μετά από κάθε νέα εγγραφή από τον ARM,

διασφαλίζοντας έτσι την ομαλή και αξιόπιστη λειτουργία του φίλτρου.

Κώδικας control-unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity control_unit is
    Port(
        valid_in: in std_logic;
        clk: in std_logic;
        rst: in std_logic;

        en: out std_logic;
        we: out std_logic;
        mac_init: out std_logic;
        rom_addr: out std_logic_vector(2 downto 0);
        ram_addr: out std_logic_vector(2 downto 0);
        valid_out: out std_logic;
        mac_en: out std_logic
    );
end control_unit;

architecture Behavioral of control_unit is

    signal count: std_logic_vector(2 downto 0) := "000";
    signal enabled_flag: std_logic := '0';

begin
```

```

process(clk, rst, valid_in)
begin
    if rst = '1' then
        count <= "000";
        mac_init <= '1';
        valid_out <= '0';
        enabled_flag <= '0';
        en <= '0';

    elsif (enabled_flag = '0' and valid_in = '1') then
        enabled_flag <= '1';
        en <= '1';

    elsif (rising_edge(clk) and enabled_flag = '1') then

        if valid_in = '1' and count = "000" then
            count <= count + 1;
            mac_init <= '1';
            valid_out <= '1';
            mac_en<='1';

        elsif valid_in = '1' and count > "000" then
            mac_init <= '0';
            valid_out <= '0';
            count <= "000";
            mac_en<='1';

        elsif valid_in = '0' and count ="000" then
            mac_init <= '0';
            valid_out <= '1';
            count <= "000";
            mac_en<='0';
        else
            mac_init <= '0';
            valid_out <= '0';
            count <= count + 1;
            mac_en<='1';
        end if;

    end if;

end process;

we <= valid_in;
rom_addr <= count;
ram_addr <= count;

end Behavioral;

```

Στη μονάδα ελέγχου πραγματοποιήθηκαν τροποποιήσεις στη λογική λειτουργία, με στόχο την ορθή διαχείριση εισόδων που απέχουν χρονικά περισσότερο από 8 κύκλους ρολογιού, όπως συμβαίνει στην περίπτωση σημάτων που προέρχονται από τον ARM. Παράλληλα, προσαρμόστηκε η λειτουργία του σήματος valid_out, ώστε να παραμένει ενεργό από τη στιγμή που το FPGA παράγει έγκυρο αποτέλεσμα μέχρι τη λήψη του επόμενου valid_in από τον ARM. Με τον τρόπο αυτό διασφαλίζεται ότι ο ARM θα διαβάσει αξιόπιστα το valid_out.

Κώδικας MAC

Για την υποστήριξη της παραπάνω τροποποιημένης λογικής, εισαγάμε ένα επιπλέον σήμα ελέγχου λογικής enable στη μονάδα MAC, όπως φαίνεται και στον παρακάτω κώδικα. Το σήμα αυτό ελέγχεται από την control unit και επιτρέπει την ενεργοποίηση ή απενεργοποίηση της MAC.

Πιο συγκεκριμένα, η MAC απενεργοποιείται μόλις ολοκληρωθεί ο υπολογισμός της τρέχουσας πράξης και παραμένει σε αυτή την κατάσταση μέχρι την άφιξη ενός νέου valid_in, έτσι το σωστό αποτέλεσμα παραμένει στην έξοδο και το valid_out μπορεί να παραμένει σωστά ενεργό. Όταν όμως φτάσει το νέο valid_in σηματοδοτείτε η έναρξη νέας πράξης, υποδεικνύοντας ταυτόχρονα ότι ο ARM έχει διαβάσει επιτυχώς το προηγούμενο αποτέλεσμα έτσι το valid_out πέφτει (μέσω της CU) και η MAC ενεργοποιείται ξανά ξεκινώντας την καινούργια πράξη. Με τον τρόπο αυτό επιτυγχάνεται σωστός συγχρονισμός μεταξύ ARM και FPGA. (Αξίζει να σημειωθεί ότι η παραπάνω διαδικασία έχει υλοποιηθεί με τέτοιο τρόπο, ώστε στην περίπτωση όπου τα δεδομένα εισόδου καταφθάνουν ανά 8 κύκλους ρολογιού, να τηρούνται πλήρως οι προδιαγραφές της προηγούμενης εργαστηριακής άσκησης, πιο συγκεκριμένα, το FIR φίλτρο σε αυτή την περίπτωση θα παράγει νέο αποτέλεσμα κάθε 8 κύκλους, χωρίς καμία επίπτωση στη λειτουργικότητά του)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.std_logic_unsigned.all;

entity mac is
    Port (
        a: in std_logic_vector (7 downto 0);
        b: in std_logic_vector (7 downto 0);
        clk: in std_logic;
        mac_init: in std_logic;

        y: out std_logic_vector (18 downto 0);
        mac_en : in std_logic
    );
end mac;

architecture Behavioral of mac is

    signal accumulator: std_logic_vector(18 downto 0) := (others => '0');

begin

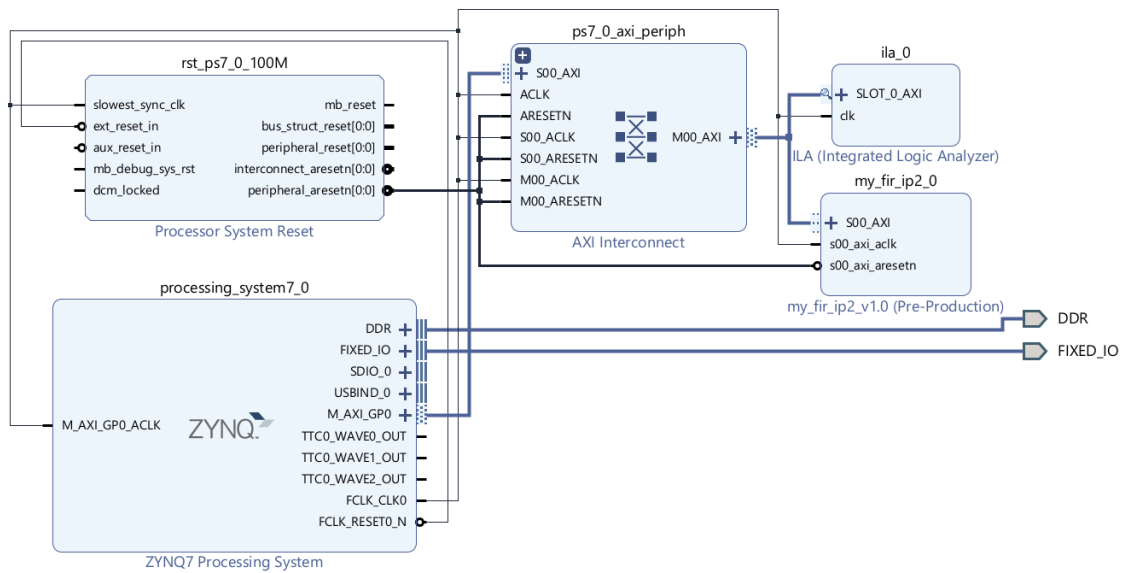
    process(clk)
    begin
        if rising_edge(clk) then
            if mac_en = '0' then
                accumulator<=accumulator;
            else
                if mac_init = '1' then
                    accumulator <= "000" & (a * b);
                else
                    accumulator <= accumulator + (a * b);
                end if;
            end if;
        end if;
    end process;

    y <= accumulator;
```

```
end Behavioral;
```

Υλοποίηση συστήματος και παραγωγή bitstream

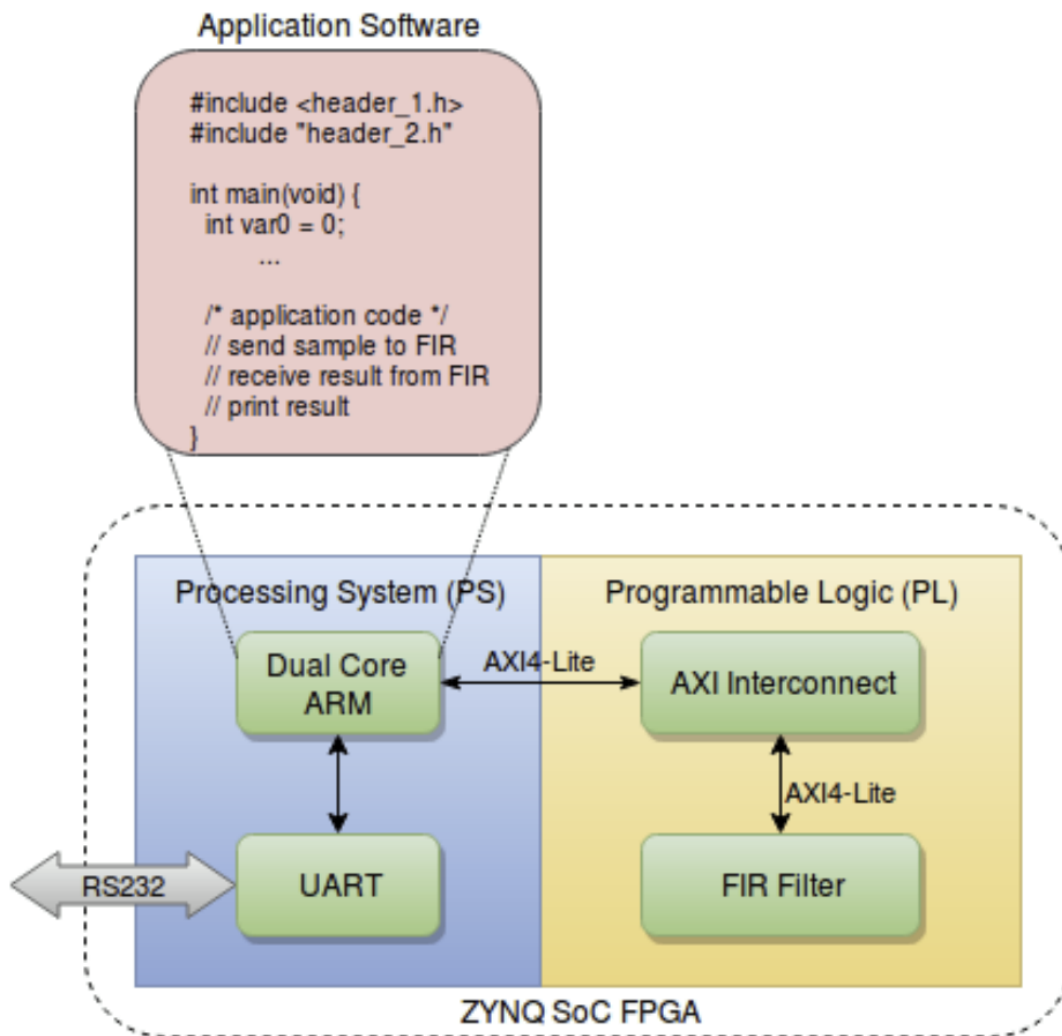
Μετά την ολοκλήρωση της υλοποίησης του φίλτρου, των αλλαγών στον κώδικα του AXI αλλά και την ενσωμάτωση του φίλτρου εντός του AXI, παράγουμε το package του IP και αφού γίνουν οι απαραίτητες διασυνδέσεις του ZYNQ με τη διεπαφή AXI4-Lite αλλά και το ILA, προκύπτει το εξής block design:



Έτσι μπορεί πλέον να παραχθεί το Bitstream με την βοήθεια του vivado και στη συνέχεια να γίνει Export Hardware με την επιλογή "Include Bitstream". Η διαδικασία αυτή έχει αποτέλεσμα τη δημιουργία ενός αρχείου .xsa, το αρχείο αυτό στην συνέχεια μπορούμε να το εισάγουμε στο Vitis, μέσω του οποίου είναι δυνατός ο συνολικός προγραμματισμός του ZYBO, τόσο του επεξεργαστή ARM όσο και του FPGA.

Προγραμματισμός του ZYNQ SoC FPGA και εκτέλεση της εφαρμογής

Παρακάτω παρουσιάζεται αρχικά η συνολική αρχιτεκτονική του συστήματος μας ,ενώ στην συνέχεια παρατίθεται ο κώδικας που υλοποιήθηκε για τον arm ώστε να πραγματοποιηθεί η επικοινωνία με το FPGA. Πιο συγκεκριμένα ο κώδικας αυτός είναι υπεύθυνος για την αποστολή των δεδομένων και στην συνέχεια για την παραλαβή των αποτελεσμάτων.



Εικόνα 2 : Αρχιτεκτονική συστήματος.

```

#include <stdio.h>
#include "platform.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "xil_io.h"
#include "xil_types.h"
#include "sleep.h"

#define FIR_BASEADDR    XPAR_MYIP_FINAL_O_SOO_AXI_BASEADDR
#define FIR_IN_OFFSET   0x0
#define FIR_OUT_OFFSET  0x4

int main()
{
    init_platform();

    u8 inputs[] = {107, 90, 224, 153, 54, 132, 130, 78, 48, 145, 151, 222, 164, 168, 155, 209,
                    160, 243, 158, 102, 0, 0, 0, 0, 0, 0, 0, 0};

    u32 receive;
    u32 word;

    Xil_Out32(FIR_BASEADDR + FIR_IN_OFFSET, (1 << 9));
    usleep(100000);
    Xil_Out32(FIR_BASEADDR + FIR_IN_OFFSET, (0 << 9));

    printf("reset\r\n");
    for (int i = 0; i < 27; i++)
    {
        printf("iteration = %d\r\n", i);

        word = (1 << 8) | inputs[i];

        Xil_Out32(FIR_BASEADDR + FIR_IN_OFFSET, word);

        do {
            receive = Xil_In32(FIR_BASEADDR + FIR_OUT_OFFSET);
            usleep(20000);

        } while (((receive >> 19) & 0x1) == 0);

        u32 result = receive & 0x7FFFF;

        printf("in = %3u -> result = %u\r\n", inputs[i], result);
    }

    printf("Finished\r\n");

    cleanup_platform();
    return 0;
}

```

Τα σήματα εισόδου και οι αντίστοιχες έξοδοι του φίλτρου παρουσιάζονται παρακάτω:

$x = \{107, 90, 224, 153, 54, 132, 130, 78, 48, 145, 151, 222, 164, 168, 155, 209, 160, 243, 158, 102, 0, 0, 0, 0, 0, 0\}$

$y = \{107, 304, 625, 1199, 1860, 2731, 3720, 4786, 4468, 4103, 3949, 3886, 4515, 5138, 5934, 6715, 6906, 7037, 7275, 7046\}$

Οι συντελεστές του FIR φίλτρου που χρησιμοποιήθηκαν είναι οι εξής:

$$h = \{1, 2, 3, 4, 5, 6, 7, 8\}$$

Τα αποτελέσματα φαίνονται παρακάτω, όπου επαληθεύουν την ορθή υλοποίηση του φίλτρου μας:

```
reset
iteration = 0
in = 107 -> result = 107
iteration = 1
in = 90 -> result = 304
iteration = 2
in = 224 -> result = 725
iteration = 3
in = 153 -> result = 1299
iteration = 4
in = 54 -> result = 1927
iteration = 5
in = 132 -> result = 2687
iteration = 6
in = 130 -> result = 3577
iteration = 7
in = 78 -> result = 4545
iteration = 8
in = 48 -> result = 4598
iteration = 9
in = 145 -> result = 4842
iteration = 10
in = 151 -> result = 3941
iteration = 11
in = 222 -> result = 3677
iteration = 12
in = 164 -> result = 4315
iteration = 13
in = 168 -> result = 4365
iteration = 14
in = 155 -> result = 4456
iteration = 15
in = 209 -> result = 5094
iteration = 16
in = 160 -> result = 6084
iteration = 17
in = 243 -> result = 6396
iteration = 18
in = 158 -> result = 6667
iteration = 19
in = 102 -> result = 6250
iteration = 20
```

```
in = 0 -> result = 6133
iteration = 21
in = 0 -> result = 5816
iteration = 22
in = 0 -> result = 5448
iteration = 23
in = 0 -> result = 4439
iteration = 24
in = 0 -> result = 3662
iteration = 25
in = 0 -> result = 1978
iteration = 26
in = 0 -> result = 816
iteration = 27
in = 0 -> result = 0
Finished
```